Implementation Plan

GGIS PropInsight

Accommodation Intelligence Application (AIA)

A build-oriented delivery plan for the location intelligence web platform — sequencing data pipelines, scoring, APIs, and UI across a four-phase roadmap, with an FCT (Abuja) pilot focus.

Prepared by: GIS & Remote Sensing Department
Geoinfotech Resources Limited
July 2026 | Version 1.0

Companion to: GGIS PropInsight Project Overview v1.2 & Technical Design Document v1.1

0. Guiding Constraints

These constraints are lifted directly from the Overview and Technical Design Documents and shape the entire build order and sequencing that follows.

- **Data pipeline before UI promise.** Nothing ships to users without a real pipeline behind it. This dictates build order: *data* → *scoring* → *API* → *UI*, not the reverse.
- **Tier discipline.** MVP ships Tier 1 domains fully automated (flood, terrain, accessibility, amenity proximity). Tier 2/3 (security, tenure, market, livability) ship with confidence badges or in later phases.
- **Flood science stays in GGIS Flood Watch.** AIA never re-derives flood risk — it consumes the GGIS API exclusively. This is a hard, critical-path integration dependency, so the contract must be locked early.
- **Modular monolith at MVP.** Split into independent services only where scaling or ownership demands it. Do not over-engineer microservices in Phase 1.
- **Pilot = FCT / Abuja only.** Every 'assemble data' task is scoped to the Federal Capital Territory.

1. Repository & Project Scaffolding

Weeks 1–2. Recommended monorepo layout (retaining the internal **AIA** codename for engineering artefacts, per Overview §1.1):

```
aia/
  apps/web/       # React 18 + TS + Vite + MapLibre + Tailwind (PWA)
  apps/admin/     # Data-ops & review moderation console
  services/api/   # FastAPI modular monolith (services as internal modules)
    location_intelligence/ scoring/ accessibility/ reports/
    ai_assistant/ community/ gateway/
  etl/          # Python geoprocessing DAGs (Celery), QGIS/GDAL
  db/           # Alembic migrations, PostGIS schema, seed data
  infra/       # docker-compose, k8s manifests (later), CI config
  packages/shared/ # shared TS types generated from OpenAPI
```

Foundation tasks

- **docker-compose dev stack:** PostgreSQL 16 + PostGIS 3.4, Redis, TiTiler, martin, FastAPI app, OSRM, Valhalla.
- **GitHub Actions:** lint + type-check (ruff/mypy, eslint/tsc) → unit tests → build images → integration tests against ephemeral PostGIS.
- **Alembic** wired up; scoring-profile changes enforced as reviewed migrations.
- **OpenAPI → TS types:** auto-generate the schema and emit typed client into packages/shared.

2. Phase 1 — Foundation (Months 1–3)

The critical-path phase. Four workstreams run in parallel.

2a. GGIS Flood Watch API contract — start day 1, highest risk

An external dependency on another team; blocking it late will blow the schedule.

- Agree the §5.1 contract: `/v1/flood/risk`, `/history`, `/alerts/subscriptions`, `/tiles/hazard/{z}/{x}/{y}`, `/meta/model`.
- Implement HMAC service-to-service signing over TLS.
- Build the **graceful-degradation path first** (§5.3): if GGIS is unreachable, the flood domain returns 'temporarily unavailable' with a cached, timestamped last-known class.
- Stand up a **mock GGIS server** so AIA development is never blocked on the real service being ready.

2b. Data architecture & ETL

- Build PostGIS schema v1 from §6.1 (locations, poi, districts, planning_layers, scores, scoring_profiles, incidents_agg, reviews, market_samples, users/api_keys). SRID 4326, GiST on every geom, geography-cast KNN indexes, partial index on poi(category).
- Implement the **layer_version discipline** early — it is the backbone of cache invalidation (Celery sweep on every layer bump).
- **OSM roads & POIs:** Geofabrik Nigeria extract → osm2pgsql → QA rules → layer bump → OSRM/Valhalla graph rebuild.
- **DEM & terrain:** SRTM/Copernicus (+ UAV DSM) → slope, flow accumulation, TWI via GDAL/rasterio → COG → STAC catalogue → TiTiler.
- **GGIS hazard tile mirror:** harvest via API → COG conversion → TiTiler registration.

2c. Scoring engine

- Implement the §4.4 weighted multi-criteria model: $score_d = 100 \cdot \Sigma(w_i \cdot n_i)$, with piecewise-linear distance decay, categorical lookups, and inverted hazard-class mapping.
- **Weights live in the scoring_profiles table, not code.** Ship the fct-v1 profile.
- Emit confidence_d (High/Medium/Low) from source recency, spatial resolution, and verification status.
- Keep the composite 'AIA Index' deliberately secondary — per-domain scores with evidence are the product.

2d. Clickable prototype

- Map-first React shell (MapLibre) with search bar, click-to-analyse, and a scorecard panel wired to a real `/v1/locations/analyze` returning Tier-1 domains. Deploy to staging.

Phase 1 exit criteria

- Schema v1 live; OSM/DEM ETL running; GGIS risk + tiles endpoints working (mock or real); scoring engine producing FCT scorecards; clickable prototype on staging.

3. Phase 2 — MVP / FCT Public Beta (Months 4–7)

Build the full request flow (§2.2) and ship the v1 endpoints (§7).

- **POST /v1/locations/analyze** — the core fan-out: parallel PostGIS spatial queries + Accessibility Service + live GGIS risk query → Scoring Engine → Redis cache (keyed by geohash8 + layer_versions) → JSON scorecard.
- **Accessibility Service:** OSRM table service (travel-time matrix), Valhalla isochrones (5/15/30 drive, 10/20 walk), rainy-season passability flag (road surface ∩ GGIS hazard).
- **Amenity finder:** /v1/amenities/nearby KNN queries.
- **Scorecard UI:** eight domain cards with expandable evidence (indicator values, data dates, confidence), tier badges, and 'why this score' explanations.
- **Report Service:** WeasyPrint HTML→PDF, queued via Celery, with static map composites, evidence tables, methodology note, and QR back to the live scorecard.
- **Auth:** OAuth2 password + refresh, JWT for users; role-based (user/moderator/admin/enterprise).
- **Observability baseline:** Prometheus/Grafana golden signals, Sentry, structured JSON logs with request IDs propagated into GGIS calls; data-freshness dashboards.
- **PWA:** cached basemap tiles + last-viewed scorecards for low-connectivity use.

Performance targets to validate (§10)

- Cached scorecard < 400 ms p95; cold (incl. GGIS live call) < 2.5 s p95; 500 sustained / 2,000 peak users. Heavy estate-AOI polygon analyses run async with progress polling.

4. Phase 3 — Community & Market Layers (Months 8–10)

- **Community & reviews (§4.9):** geotagged reviews, phone-verification + moderation queue, district-level aggregation only; raw security-sensitive reports never shown.
- **Security domain (Tier 2–3):** incidents_agg district-level aggregates from curated sources (ACLED etc.) + police proximity. No address-level crime mapping (§8 ethics constraint).
- **Market price surfaces:** geocoded listings/transactions → outlier filtering → kriging/IDW surfaces.
- **Compare mode:** up to four pinned locations, side-by-side matrix.
- **Alert subscriptions:** proxy GGIS webhooks → Celery fan-out (email/SMS/push).
- **Payments & premium gating; enterprise API keys** with per-tier quotas.

5. Phase 4 — Scale & Enterprise (Months 11–15)

- **/v1/batch/analyze** portfolio screening for banks/insurers; metered enterprise surface with usage accounting + webhooks.
- **K8s migration:** promote services out of the monolith; PostGIS read replicas; dedicated OSRM/Valhalla nodes; multi-AZ.
- **Data onboarding** for Lagos + 2–3 additional states.
- **Mobile app packaging.**

6. Cross-Cutting Decisions to Make Now

Decision	Recommendation & rationale
Hosting	Prototype on a Railway-style PaaS for speed; commit to **GCP** (aligns with existing Geoinfotech cloud experience) or AWS before public beta for managed PostGIS, CDN, and object-storage maturity.
Monolith vs services	Modular monolith through MVP; split only where scaling/ownership demands it (§1.4).
Tiles	martin (vector, from PostGIS) + TiTiler (dynamic COG raster); CDN in front.
AI assistant	Claude API with tool-use grounded **strictly** on AIA endpoints — must cite returned data, refuse out-of-coverage questions. Defer to late Phase 2 / Phase 3.
Privacy	NDPR-aligned; minimal PII on reviews; store phone-verification hashes only.

7. Top Risks to Manage Actively

#	Risk	Mitigation
1	GGIS Flood Watch API readiness — external-team dependency on the critical path.	Lock the contract in Phase 1 and develop against a mock server so AIA is never blocked.
2	FCT data completeness — success metric is ≥ 90% district coverage at launch.	Front-load the data audit; ETL QA rules and field-verification flags are what get you there.
3	Tenure/legal liability.	Keep tenure strictly advisory with dated sources and confidence labels; never present it as a legal search (Tier 2–3).
4	layer_version cache correctness.	Get the invalidation sweep right early, or scorecards will silently serve stale data.

8. Immediate Next Steps (First Two Weeks)

- Lock the GGIS Flood Watch API contract with that team and stand up the mock server.
- Scaffold the monorepo, docker-compose stack, and CI/CD.
- Land PostGIS schema v1 + Alembic + the layer_version mechanics.
- Kick off the FCT OSM + DEM ETL and author the fct-v1 scoring profile.

This plan operationalises the phased roadmap in the Project Overview (§8) and the engineering milestones in the Technical Design Document (§12), scoped to the FCT pilot. It is intended to be sufficient for the development team to begin Phase 1 execution.